LECTURE 4

# MACHINE LEARNING ON DATA REPRESENTATIONS

Learning on fixed representations

PRESS → OR SPACE.

© Marc Rußwurm
Licensed under **Creative Commons Attribution–NonCommercial 4.0 International (CC BY-NC 4.0)**.
You may share and adapt these slides for teaching, research, and other non-commercial educational purposes with attribution.
Commercial training, paid workshops, consulting seminars, or incorporation into commercial course products require prior permission.

UNIVERSITÄT BONN

# Learning Outcome Roadmap

| | Learning outcome | Lecture | Block |
|---|---|---|---|
| ⬜ | Explain how heterogeneous datasets align to a common coordinate support and form a geospatial data cube. | Lecture 3 | Geospatial Dat |
| ⬜ | Represent locations as fixed feature vectors extracted from a geospatial data cube. | Lecture 3 | Geospatial Dat |
| ⬜ | Explain how linear regression maps a fixed representation to a continuous target. | Lecture 4 | Linear Models |
| ⬜ | Explain how logistic regression maps a fixed representation to categories with a linear decision boundary. | Lecture 4 | Linear Models |
| ⬜ | Explain how decision trees partition a fixed representation space into nonlinear decision regions. | Lecture 4 | Decision Trees |
| ⬜ | Explain how random forests combine trees for robust predictions from fixed representations. | Lecture 4 | Decision Trees |

# Linear Regression

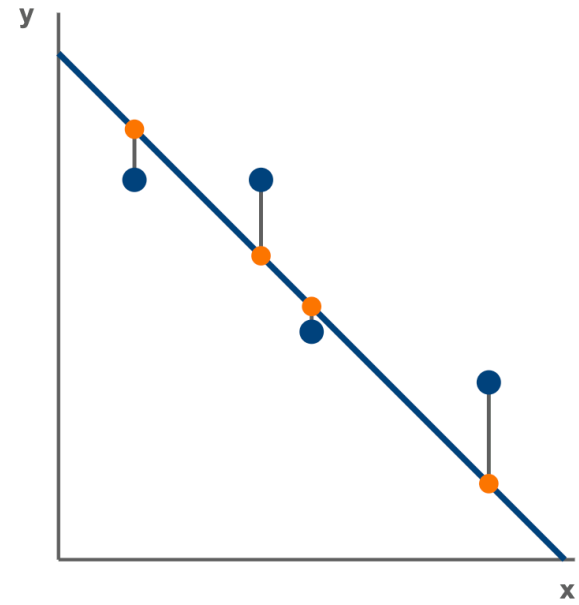



Figure: M. Rußwurm & D. Tuia, *Linear Regression*, ENV-408 course materials, EPFL, 2023.

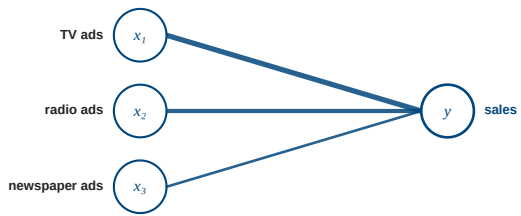
# Why Linear Regression?

- a simple, classical approach to supervised learning
- predicts a **continuous** response from one or more predictors
- around for a long time, yet still one of the most used tools in science
- a strong, interpretable baseline — and a stepping stone to more complex models

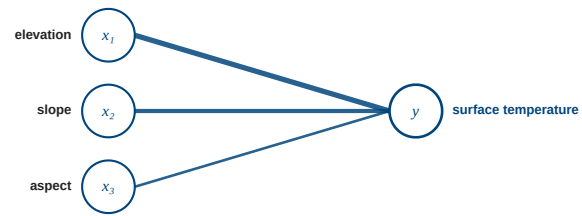
**Learning linear regression helps understand more complex methods, and it is practically useful and easy to implement for many real-world problems.**

# Measuring Relationships Between Variables

### Classic example



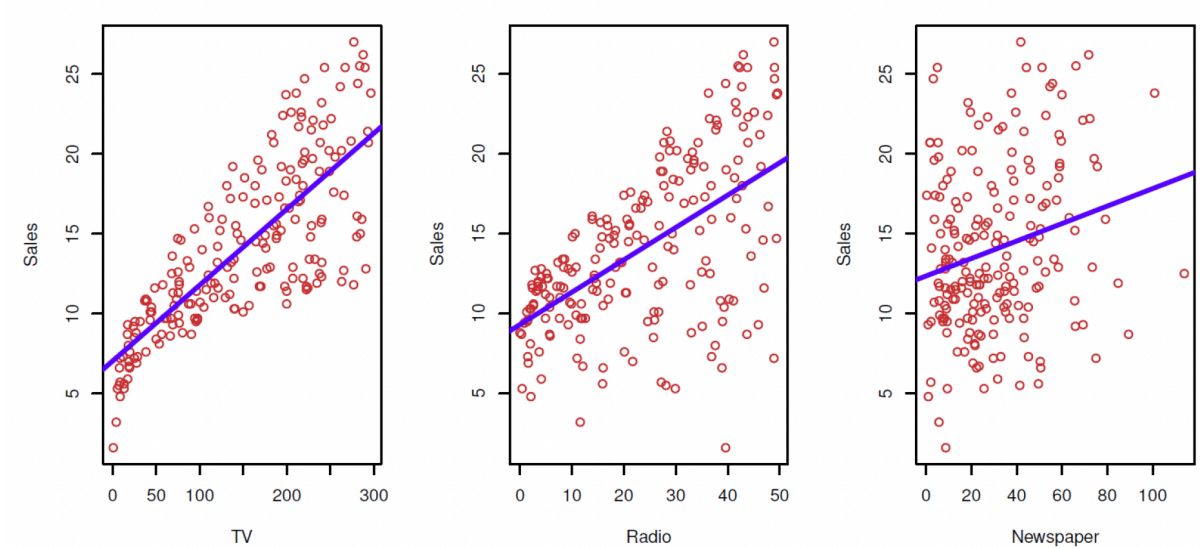

### Geospatial example




Linear regression models how one or more predictor variables (features) relate to a continuous target — exactly the fixed representations built in the previous section.

# A Closer Look at the Data

$\mathcal{D} = \{((x_{1,i}, x_{2,i}, x_{3,i}), y_i)\}_{i=1}^{N}$




$x_1 \mapsto y$ (TV → sales)

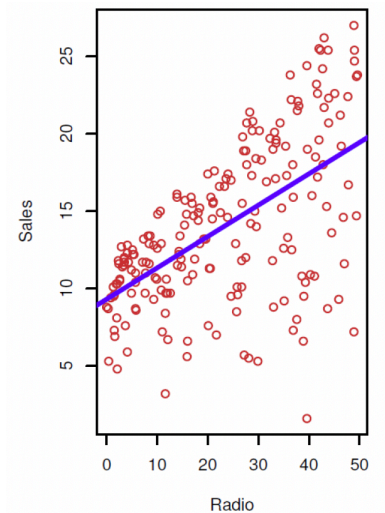
$x_2 \mapsto y$ (radio → sales)

$x_3 \mapsto y$ (newspaper → sales)

Figure: James, Witten, Hastie & Tibshirani, *An Introduction to Statistical Learning*, Advertising dataset.

# Linear Regression Answers Important Questions

- is there a relationship between advertising budget and sales?
- how strong is the relationship?
- is there synergy among the advertising media?




The same questions apply to our geospatial example: is there a relationship between elevation and surface temperature — and how strong is it?

# Univariate Linear Regression

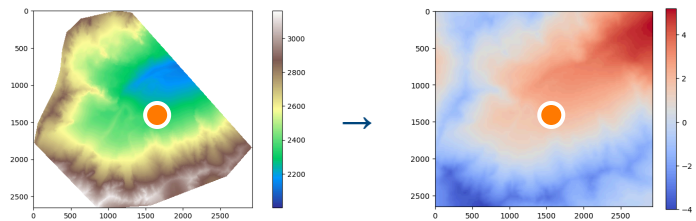


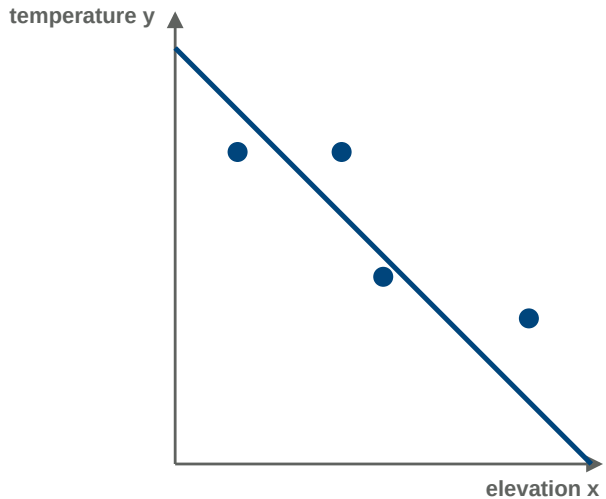

Figure: M. Rußwurm & D. Tuia, *Linear Regression*, ENV-408 course materials, EPFL, 2023.

A simple (but powerful) linear model

$$\hat{y} = f(x; \beta_0, \beta_1) = \beta_0 + x\beta_1$$

maps an elevation $x$ to an estimated temperature $\hat{y}$, with $\beta_0$ the y-intercept and $\beta_1$ the slope.

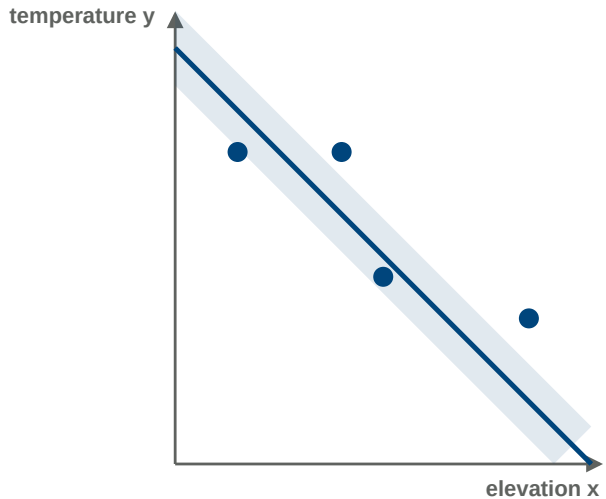
# The Problem: Finding $\hat{\beta}$




We have a dataset of data samples $(x_i, y_i)$.

**Problem: how do we find good model parameters $\hat{\boldsymbol{\beta}} = (\hat{\beta}_0, \hat{\beta}_1)$ for our linear model $f(x; \hat{\boldsymbol{\beta}}) = \hat{\beta}_0 + x\hat{\beta}_1$?**

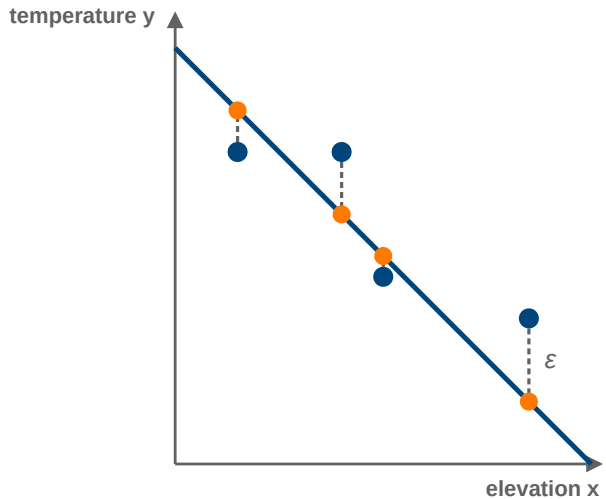
# What Is a Good Model — Formally?




$$y_i = f(x_i; \hat{\boldsymbol{\beta}}) \pm \overbrace{\varepsilon}^{\text{error}}$$
$$\hat{y}_i = f(x_i; \hat{\boldsymbol{\beta}})$$

with the error $\varepsilon$ as a random variable — the model never fits the data perfectly.

# Residuals



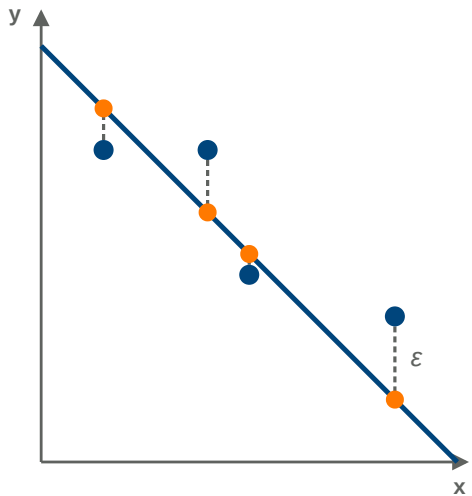

Concretely, the error — the "residual" $\varepsilon_i$ — for each sample $i$:

$$|\hat{y}_i - y_i| = \varepsilon_i$$

We want to minimize the residuals over the whole dataset

$$\min \frac{1}{N} \sum_{i=1}^{N} \varepsilon_i$$

# Mean Absolute Error (MAE)

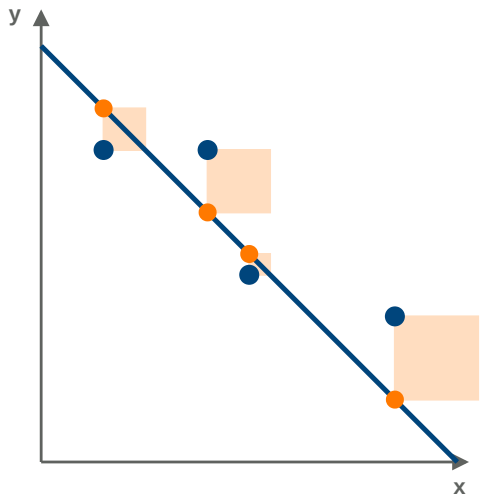



$$\mathrm{MAE}(\mathcal{D}, \boldsymbol{\beta}) = \frac{1}{N} \sum_{i=1}^{N} |\hat{y}_i - y_i|$$

over model $\boldsymbol{\beta}$ with $\hat{y}_i = f(x_i; \boldsymbol{\beta})$ and dataset $\mathcal{D} = \{x_i, y_i\}_{i=1}^{N}$

**Very intuitive — but does not punish large errors.**

# Mean Squared Error (MSE)




$$\mathrm{MSE}(\mathcal{D}, \boldsymbol{\beta}) = \frac{1}{2N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2$$

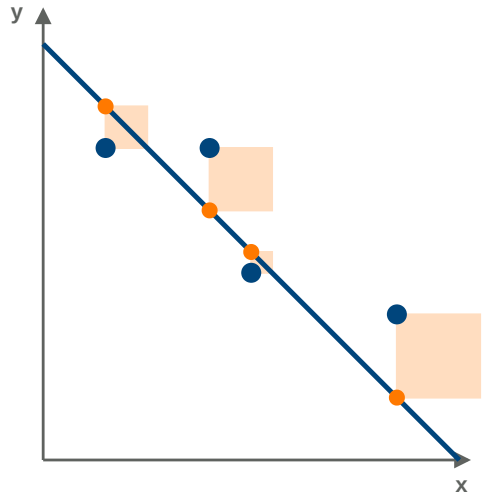
Punishes large errors with a square penalty.

**Geometric intuition: find a model that minimizes the total area of these squares.**

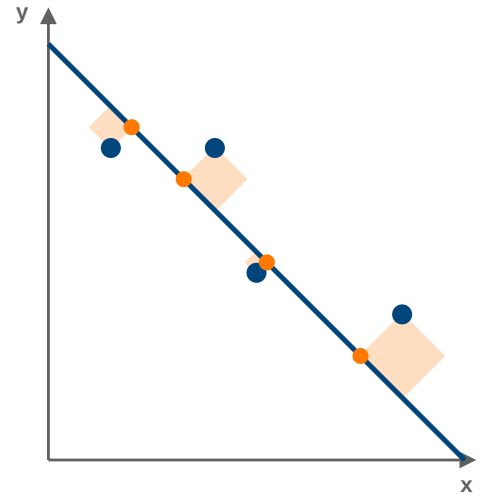
# Vertical vs. Perpendicular Error Offsets

Both measure a residual — but they disagree on what "error" means for a spatial variable.

**Vertical offsets**




**Perpendicular offsets**



# Ordinary Least Squares (OLS) — Intuition

The closed-form solution $\hat{\beta}_0, \hat{\beta}_1$ for a linear model under MSE.

**Task: find $\hat{\beta}_0, \hat{\beta}_1$ that minimize the Mean Squared Error**

$$\hat{\beta}_0, \hat{\beta}_1 = \arg\min_{\beta_0, \beta_1} \frac{1}{2N} \sum_{i=1}^{N} \left( \overbrace{\beta_0 + x_i \beta_1}^{\hat{y}_i} - y_i \right)^2$$

**Question:** how do we minimize a function with respect to its parameters?

- calculate the partial derivatives $\frac{\partial \text{MSE}}{\partial \beta_0}$ and $\frac{\partial \text{MSE}}{\partial \beta_1}$
- set the derivatives to zero: $\frac{\partial \text{MSE}}{\partial \beta_0} = 0$, $\frac{\partial \text{MSE}}{\partial \beta_1} = 0$
- solve for $\beta_0$ and $\beta_1$

# Ordinary Least Squares (OLS) — Solution

Model $y = \beta_0 + x\beta_1$ on a dataset $\mathcal{D} = \{x_i, y_i\}_{i=1}^N$

Ordinary Least Squares objective

$$\hat{\beta}_0, \hat{\beta}_1 = \arg\min_{\beta_0, \beta_1} \frac{1}{2N} \sum_{i=1}^{N} \left( \overbrace{\beta_0 + x_i \beta_1}^{\hat{y}_i} - y_i \right)^2$$

Closed-form solution

$$\hat{\beta}_0 = \frac{1}{N} \sum_{i=1}^{N} y_i - \hat{\beta}_1 \frac{1}{N} \sum_{i=1}^{N} x_i = \bar{y} - \hat{\beta}_1 \bar{x}$$

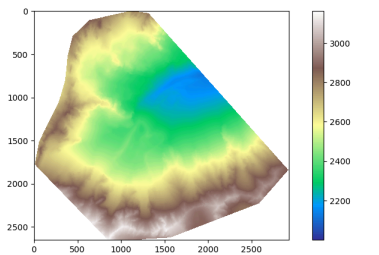
$$\hat{\beta}_1 = \frac{\operatorname{Cov}[x, y]}{\operatorname{Var}[x]} = \frac{\frac{1}{N} \sum_i (x_i - \bar{x})(y_i - \bar{y})}{\frac{1}{N} \sum_i (x_i - \bar{x})^2}$$

See **Ordinary Least Squares**, Wikipedia.

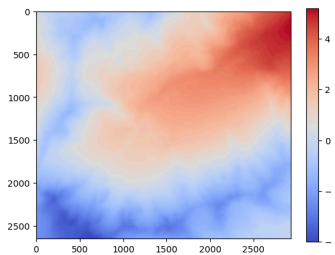
# Limits of Univariate Models

Some relationships are well-described by a single variable — most are not.

## Roughly linear

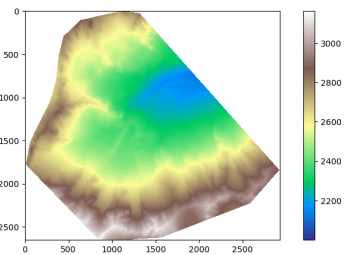


→

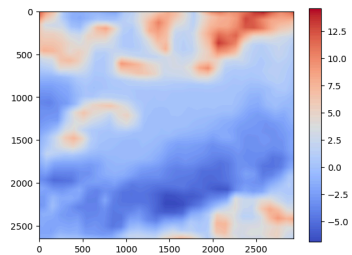


elevation → average temperature, roughly $-0.006 \frac{^\circ\mathrm{C}}{\mathrm{m}}$

## Clearly non-linear

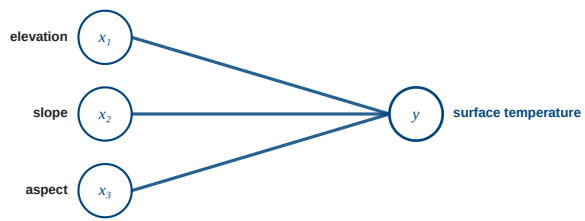


?



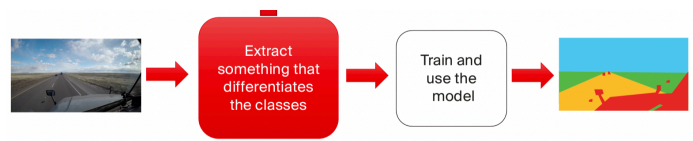
elevation → surface temperature — shading, wind, and local terrain also matter

# Multivariate Models: Linearize With Good Feature Design




Many non-linear relationships can be linearized with good feature design: elevation alone under-explains surface temperature, but elevation **and** slope **and** aspect together explain it much better.

Same pattern as classical feature extraction: extract informative features, then fit a simple model.



# From Uni- to Multivariate Linear Models

**Univariate model** $y = f(x; \beta_0, \beta_1)$ with scalar $x, y$ and two parameters

$$y = \beta_0 + \overbrace{x_1}^{\text{e.g., elev.}} \beta_1$$

**Multivariate linear model**

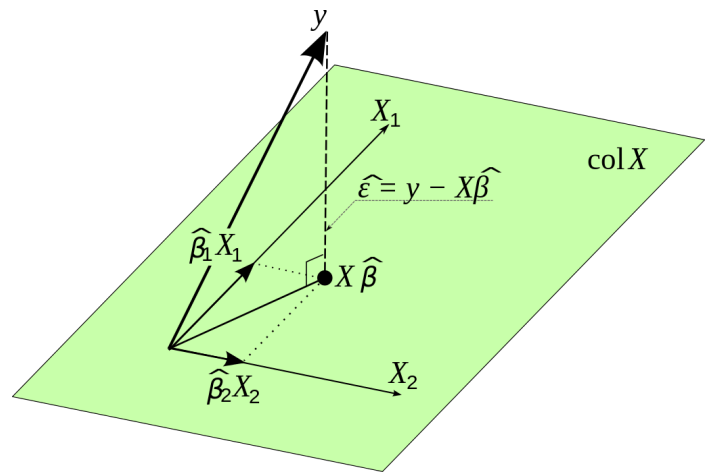
- one input sample $\mathbf{x}_i = (1, x_1, x_2, x_3, \ldots, x_p)$ of $p$ features
- corresponding target $y_i$
- one weight per feature $\boldsymbol{\beta} = (\beta_0, \beta_1, \beta_2, \ldots, \beta_p)$

$$y_i = \beta_0 + \overbrace{x_1}^{\text{e.g., elev.}} \beta_1 + \overbrace{x_2}^{\text{e.g., slope}} \beta_2 + \overbrace{x_3}^{\text{e.g., aspect}} \beta_3 + \cdots = \boldsymbol{\beta}^\top \mathbf{x}_i$$

# Multivariate Linear Model in Matrix Form

A multivariate dataset $\mathcal{D} = \{\mathbf{x}_i, y_i\}_{i=1}^n$ as matrix $\mathbf{X}$ and target vector $\mathbf{y}$, with $n$ samples and $p$ features

$$\mathbf{X} = \begin{pmatrix} 1 & \overbrace{x_{11}}^{\text{elev.}} & \overbrace{x_{21}}^{\text{slope}} & \overbrace{x_{31}}^{\text{aspect}} & \dots \\ 1 & x_{12} & x_{22} & x_{32} & \dots \\ \vdots & \vdots & \vdots & \vdots & \ddots \\ 1 & x_{1n} & x_{2n} & x_{3n} & \dots \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$




And the model

$$\mathbf{y} = \mathbf{X}\hat{\boldsymbol{\beta}} + \overbrace{\boldsymbol{\varepsilon}}^{\text{errors}}$$

with $p+1$ parameters $\hat{\boldsymbol{\beta}} = (\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p)$

$$\boldsymbol{\varepsilon} = \overbrace{\left\| \mathbf{X}\hat{\boldsymbol{\beta}} - \mathbf{y} \right\|^2}^{\text{MSE}}$$

# Multivariate Ordinary Least Squares

$$\hat{\boldsymbol{\beta}} = \arg\min_{\hat{\boldsymbol{\beta}}} \mathrm{MSE}(\hat{\boldsymbol{\beta}}) = \arg\min_{\hat{\boldsymbol{\beta}}} \left\| \mathbf{X}\hat{\boldsymbol{\beta}} - \mathbf{y} \right\|^2$$

**Question:** how do we minimize a function with respect to its parameters — now a whole vector $\hat{\boldsymbol{\beta}}$?

- calculate the partial derivatives $\frac{\partial \mathrm{MSE}}{\partial \hat{\boldsymbol{\beta}}}$
- set the derivatives to zero: $\frac{\partial \mathrm{MSE}}{\partial \hat{\boldsymbol{\beta}}} = 0$
- solve for $\hat{\boldsymbol{\beta}}$

# Multivariate OLS — Closed-form Derivation

With Mean Squared Error

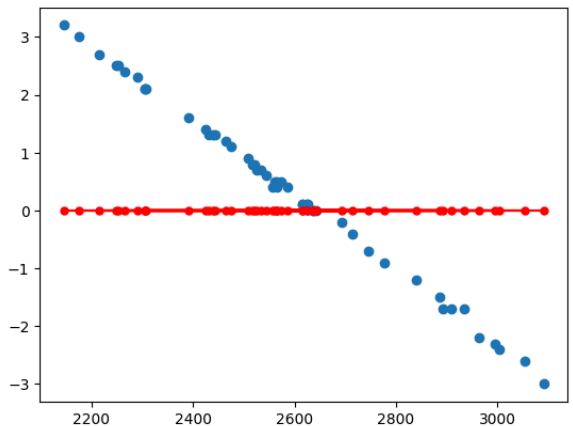
$$\mathrm{MSE}(\hat{\boldsymbol{\beta}}) = \left\|\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}\right\|^2 = \mathbf{y}^\top\mathbf{y} - 2\hat{\boldsymbol{\beta}}^\top\mathbf{X}^\top\mathbf{y} + \hat{\boldsymbol{\beta}}^\top\mathbf{X}^\top\mathbf{X}\hat{\boldsymbol{\beta}}$$

and its derivative set to zero

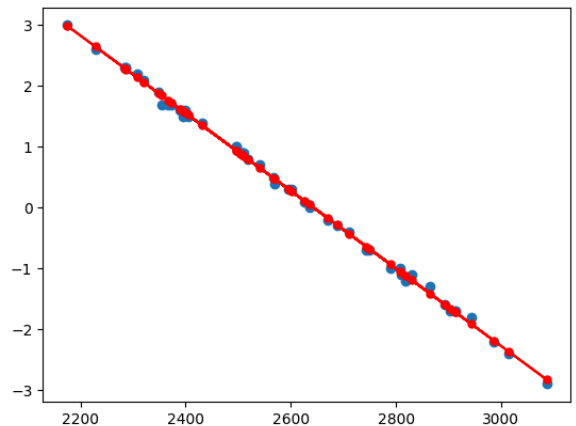
$$\begin{aligned}
\frac{\partial \mathrm{MSE}(\hat{\boldsymbol{\beta}})}{\partial \hat{\boldsymbol{\beta}}} &= -2\mathbf{X}^\top\mathbf{y} + 2\mathbf{X}^\top\mathbf{X}\hat{\boldsymbol{\beta}} = 0 \\
\mathbf{X}^\top\mathbf{X}\hat{\boldsymbol{\beta}} &= \mathbf{X}^\top\mathbf{y} \\
\hat{\boldsymbol{\beta}} &= \left(\mathbf{X}^\top\mathbf{X}\right)^{-1}\mathbf{X}^\top\mathbf{y}
\end{aligned}$$

**This is the normal equation — a closed-form solution, no iterative optimization required.**

# Let’s Appreciate the Elegance



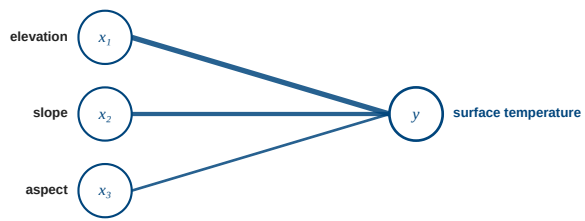

$\boldsymbol{\beta} = (0, 0, 0, 0, \dots)$




$\boldsymbol{\beta} = (-.1, .2, .5, -.5, \dots)$

$\hat{\boldsymbol{\beta}} = \left(\mathbf{X}^\top \mathbf{X}\right)^{-1} \mathbf{X}^\top \mathbf{y}$ **— a single line of code, and blazing fast.**

# Interpretable Coefficients



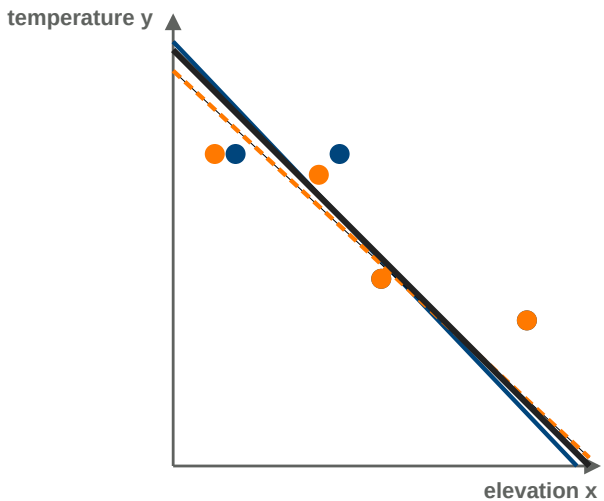

```
coefficients
beta 0 y-intercept : 9.769941
beta 1 dem         : -0.004522
beta 2 slope       : 0.032874
beta 3 dem_hor     : -0.026361
beta 4 dem_vert    : 0.222707
beta 5 aspect      : 0.002971
beta 6 diff_dem    : 0.012950
```

Each $\hat{\beta}_j$ tells us how $\hat{y}$ changes per unit of $x_j$, holding the other features fixed.

**Caveat: coefficients are only directly comparable across features when the features share units — or are normalized.**

Figure: M. Rußwurm & D. Tuia, *Linear Regression*, ENV-408 course materials, EPFL, 2023.

# Model Evaluation




We can now fit a model $(\hat{\beta}_0, \hat{\beta}_1, \dots)$ to any set of data samples $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ — but different samples give different fits.

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \varepsilon$$

**Did we capture the true, underlying relationship** $(\beta_0, \beta_1, \dots)$?

1. how large is the data error $\varepsilon$? → RSE
2. does our model explain the target variable well? → $R^2$
3. how close is $\hat{\beta}$ to the true $\beta$? → $\mathrm{SE}[\hat{\beta}]$

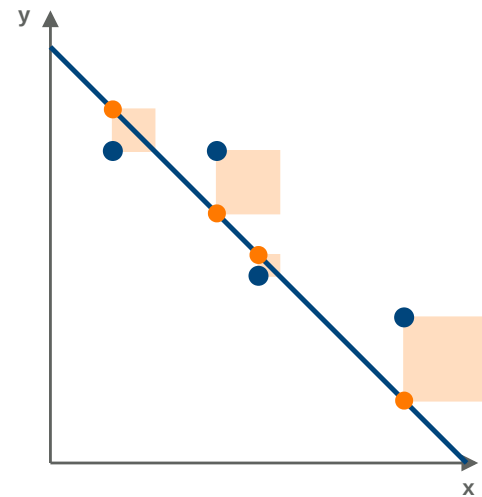
**→ a hypothesis test tells us whether the relationship is significant given the dataset size.**

# Residual Sum of Squares (RSS)

$$\mathrm{RSS} = \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

RSS increases with the dataset size $N$.

**Idea: normalize by the number of data samples.**



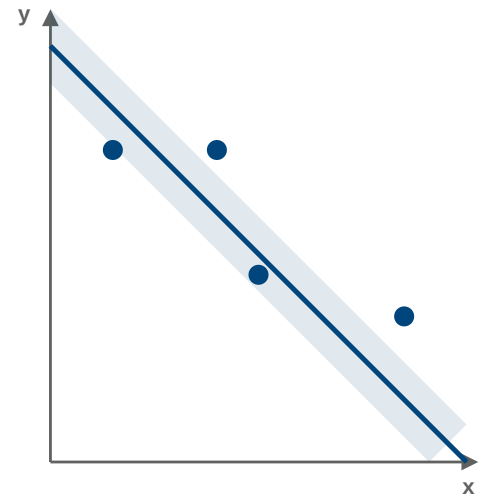
# Residual Standard Error (RSE)

$$\mathrm{RSE} = \sqrt{\underbrace{\frac{1}{N-M-1}}_{\mathrm{DoF}}\mathrm{RSS}}$$

- normalized by the number of samples $N$
- and the degrees of freedom (DoF): $M$ slope parameters and one y-intercept

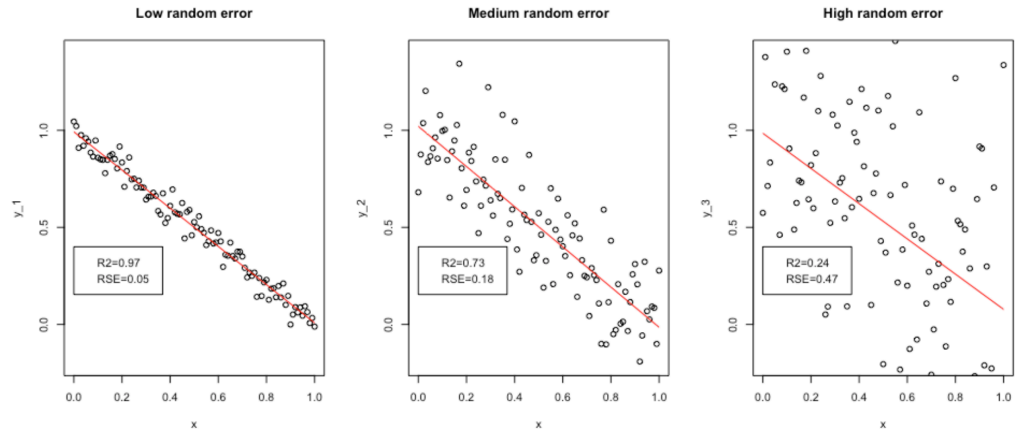
**The RSE estimates $\mathrm{std}(\varepsilon)$ and is a good measure of fit.**

**→ but it is in the units of the target variable: what number counts as a good fit?**



# R² Metric: Coefficient of Determination

The R² is a **unitless proportion**, typically between 0 and 1: it measures how much of the variability in $Y$ is explained by $\hat{Y}$.




In the univariate case, R² is the square of the **Pearson correlation coefficient** $r(Y, \hat{Y}) = \dfrac{\mathrm{Cov}[Y, \hat{Y}]}{\sigma_Y \sigma_{\hat{Y}}}$.

Figure: M. Rußwurm & D. Tuia, *Linear Regression*, ENV-408 course materials, EPFL, 2023. See also **Coefficient of Determination**, Wikipedia.

# $R^2$: Comparison to an Averaging Model

How much better is our model than a simple averaging model that always predicts $\bar{y}$, no matter the input $x$?

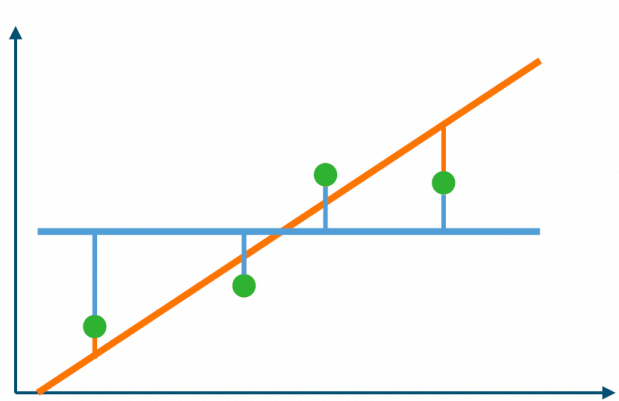
$$\mathrm{R}^2 = 1 - \frac{\mathrm{RSS}}{\mathrm{TSS}}$$

Residual Sum of Squares

$$\mathrm{RSS} = \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

Total Sum of Squares

$$\mathrm{TSS} = \sum_{i=1}^{N} (y_i - \bar{y})^2$$



# Estimating the Standard Error for Coefficients

In the underlying model

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \varepsilon$$

we can estimate the residual standard error $\sigma = \mathrm{std}(\varepsilon) = \mathrm{RSE}$ — the better the fit, the smaller the standard deviation of $\varepsilon$.

**How close are the estimated coefficients $\hat{\beta}$ to the true, underlying relationship $\beta$?**

We can calculate the Standard Error (SE) for each coefficient:

$$\mathrm{SE}[\hat{\beta}_0] = \sigma^2 \left[ \frac{1}{N} + \frac{\bar{x}^2}{\sum_{i=1}^{N} (x_i - \bar{x})^2} \right], \qquad \mathrm{SE}[\hat{\beta}_1] = \frac{\sigma^2}{\sum_{i=1}^{N} (x_i - \bar{x})^2}$$

# Student's t-test: Is Our Model Significant?

**Question:** is an estimated relationship a random artifact of the data?

Model: $Y = \beta_0 + \beta_1 X_1 + \varepsilon$

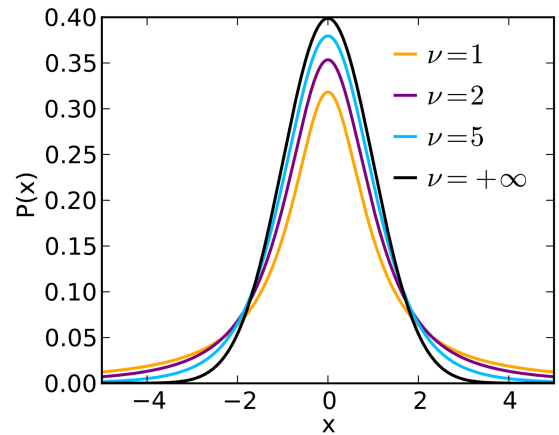
Testing the slope $\beta_1$

$$H_0 : \beta_1 = 0 \qquad H_1 : \beta_1 \neq 0$$

Test statistic

$$t_0 = \frac{\hat{\beta}_1}{\mathrm{SE}[\hat{\beta}_1]}$$

**Reject $H_0$ if $t_0 < -t_{\alpha,N-2}$ or $t_0 > t_{\alpha,N-2}$**




$t_{\alpha,N-2}$ comes from the Student-t distribution with $\nu = N - 2$ degrees of freedom.

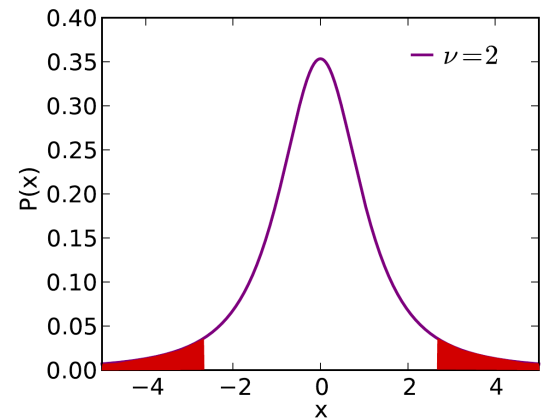
Figure: Wikipedia, author Skbkekas, Student's t-distribution. CC BY-SA.

# P-value: The Probability of a Lucky Shot

The (two-sided) p-value integrates the Student-t distribution to $t_0$ on both sides.

high p-value → not enough data

high p-value → no relationship




p-value shown in red

**We can use the p-value of different coefficients to select significant features.**

Figure modified from Wikipedia, original author Skbkekas, Student's t-distribution. CC BY-SA.

# Recap: Linear Regression

This part covered how to:

- implement a univariate and multivariate linear model from scratch
- fit the parameters with the Ordinary Least Squares (OLS) method
- evaluate the goodness of fit with RSE and $R^2$
- test individual parameters for significance with Student's t-test
- select features dynamically based on their significance

**Next: logistic regression extends the same linear model to map a fixed representation to categories with a linear decision boundary.**

# Non-linear models: Decision Trees & Random Forests



Image by Bing Image Generator

# Decision Trees: The Core Idea

We categorize data through a **chain of simple decisions**.

- each decision is a simple test on a single feature (e.g., *is the temperature below 10 °C?*)
- implemented in computer science as a **binary tree**: a graph data structure in which each node has at most two children
- a universal tool for **classification and regression** of non-linear relationships



Image by Bing Image Generator

# Recap: Data, Data Distributions & Models

**1 · collect data** of the real world: we "sample" training data from a distribution

$$\{(X_i, y_i)\}_{i=1}^{N} \sim P(X, y)$$

**3 · validate / evaluate** the model on test data sampled from the same distribution

$$\{(X_i, y_i)\}_{i=1}^{M} \sim P(X, y)$$

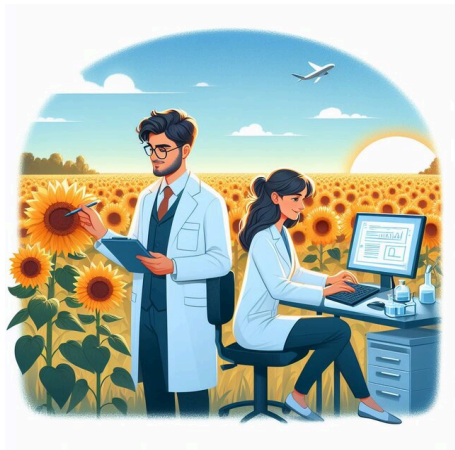


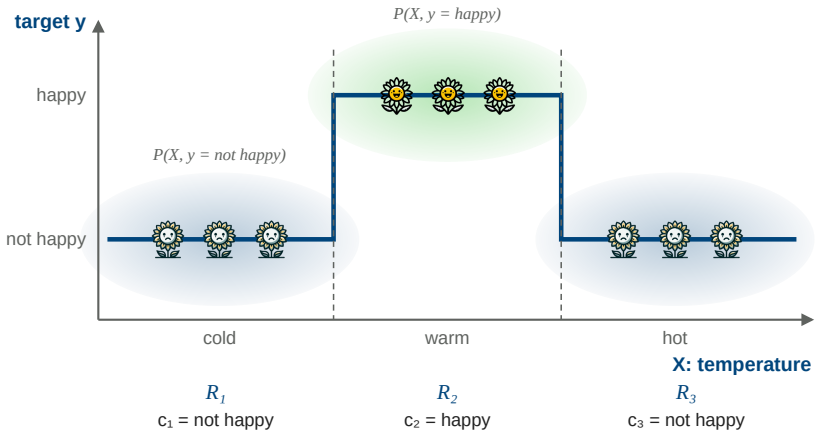
Image by Bing Image Generator

**2 · create / fit** a mathematical model that approximates the world on training data

$$\hat{y}_i = f(X_i), \qquad \min E_{\text{train}}(\hat{y}_i, y_i)$$

**We are interested in the test error $E_{\text{test}}(\hat{y}_i, y_i)$.**

# Linear Models Fail in Non-linear Situations



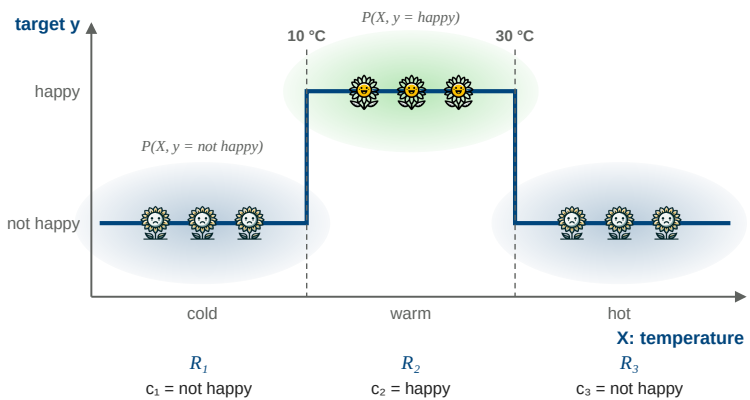

We need a different model: a **piecewise constant** function over regions $R_m$

$$f(X) = \sum_{m=1}^{M} c_m \cdot 1_{(X \in R_m)}$$

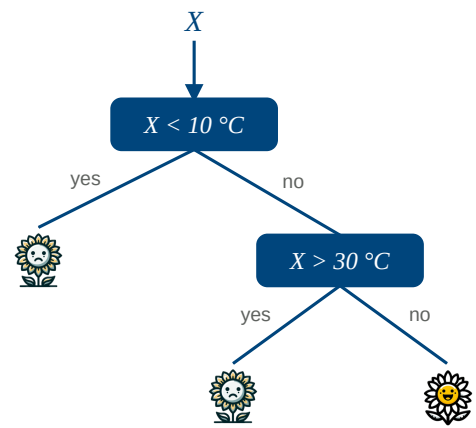
with one value $c_m$ per region.

# Decision Trees: Regions from Binary Decisions

$f(X) = \sum_{m=1}^{M} c_m \cdot 1_{(X \in R_m)}$ — regions defined by intervals: $R_1 = \{X \leq 10\,^\circ\mathrm{C}\}$, $R_2 = \{10\,^\circ\mathrm{C} < X \leq 30\,^\circ\mathrm{C}\}$, $R_3 = \{X > 30\,^\circ\mathrm{C}\}$

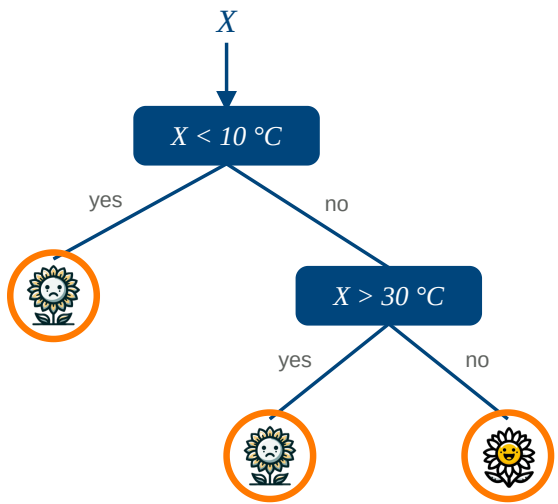



**A simple decision tree**



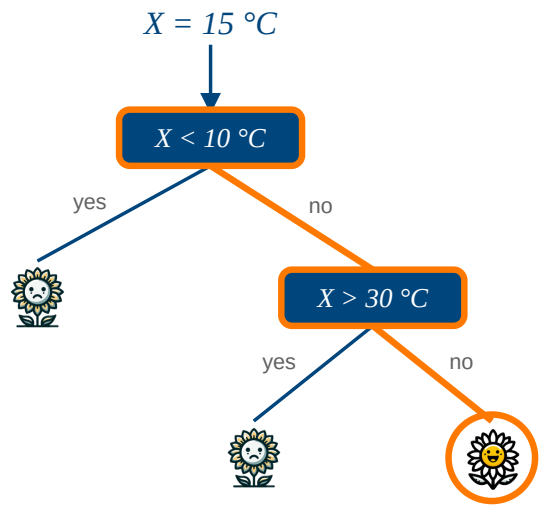

(drawn upside down: the root is at the top)

# A Classification Decision Tree: Terminology




- **input features**: the data to be classified
- the **root node** starts the decision process
- **decision nodes** (split nodes) implement a **test function with discrete outcomes**; each test splits the input space into regions
- **leaf nodes** mark the end of a sequence of decisions; a single (output) class is associated with each leaf

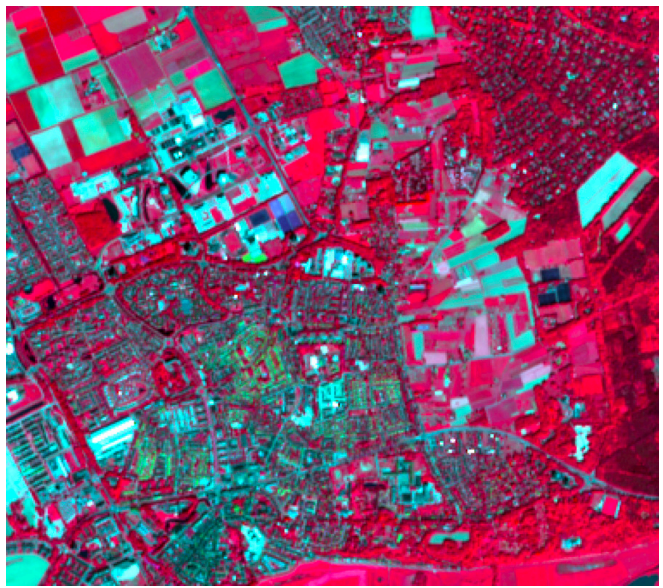
# A Simple Algorithm to Classify a Test Sample



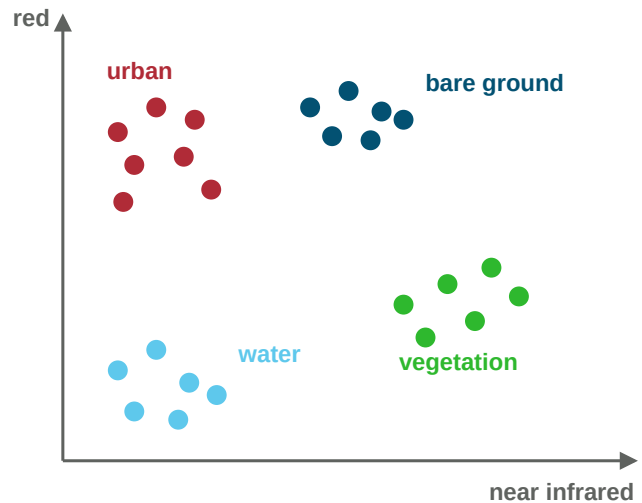

1. start at the root node
2. if the current node is a leaf node, return its class label
3. perform the test of the current decision node and follow the corresponding branch
4. go to 2

$15\,^{\circ}\mathrm{C}$: **not below** $10\,^{\circ}\mathrm{C}$**, not above** $30\,^{\circ}\mathrm{C}$ → **happy.**

# Multivariate Data Example: Land Cover Classification

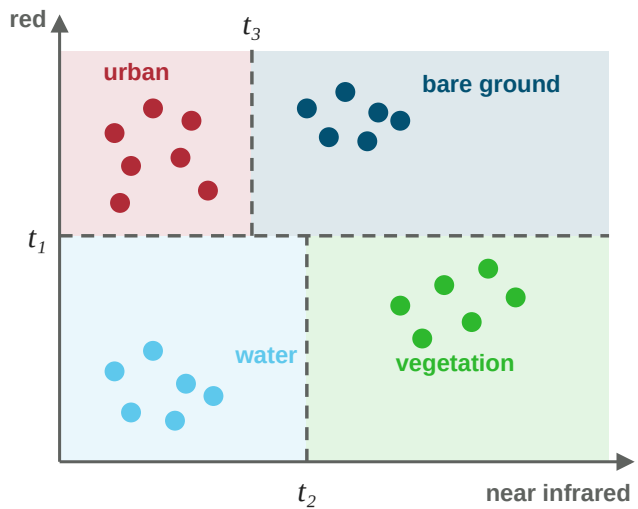


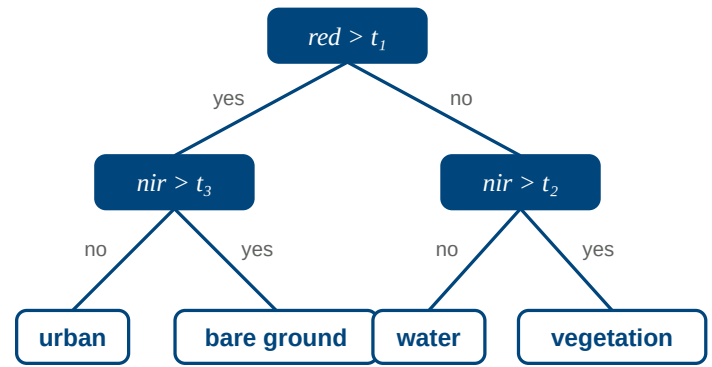
Sentinel-2, false-colour composite (Wageningen, NL). Contains modified Copernicus Sentinel data.




Each pixel is a point in feature space: its **red** and **near-infrared** reflectance.

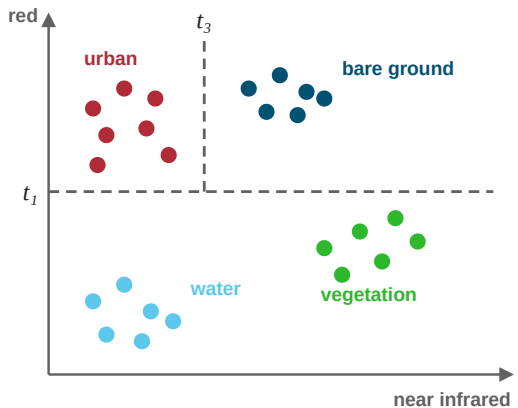
# Decision Tree for Multivariate Data

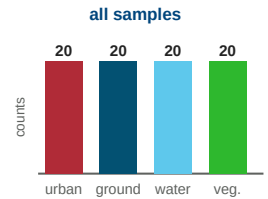


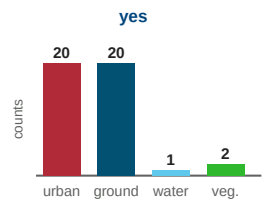

Every split is **axis-aligned**: each decision tests one feature against one threshold, so the regions are rectangles.

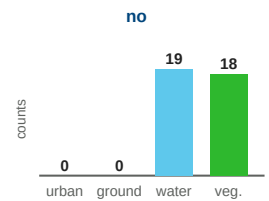
# Increasing Node Purity: Split & Repeat



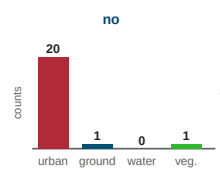

*red* > $t_1$

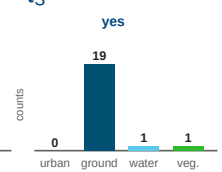



**Idea: every decision should increase the node purity.**

*nir* > $t_3$



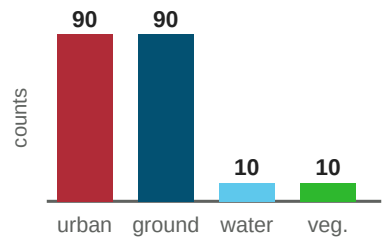




… and so on

# Quantifying Node Purity: Entropy

$$H(\mathbf{p}) = -\sum_i p_i \log(p_i) \qquad (\text{here with } \log_{10})$$

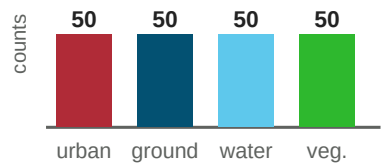
**lower entropy: more pure**




$p_1 = p_2 = \frac{90}{200} = 0.45, \quad p_3 = p_4 = \frac{10}{200} = 0.05$

$H = -(2 \cdot 0.45 \log 0.45 + 2 \cdot 0.05 \log 0.05) \approx \mathbf{0.44}$

**higher entropy: less pure**



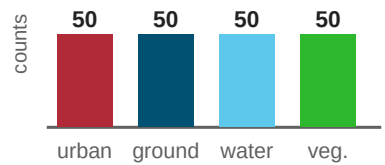

$p_{1,2,3,4} = \frac{50}{200} = 0.25$

$H = -4 \cdot 0.25 \log 0.25 \approx \mathbf{0.60}$

# Pen & Paper Exercise: Gini Index

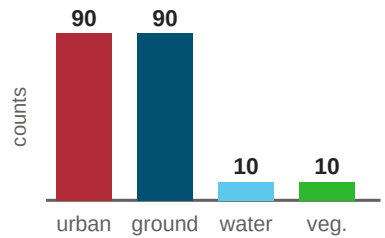
$$G(\mathbf{p}) = \sum_i p_i(1 - p_i)$$

**higher Gini index: less pure**




$G = 4 \cdot 0.25 \cdot 0.75 = \mathbf{0.75}$

**lower Gini index: more pure**




$G = 2 \cdot 0.45 \cdot 0.55 + 2 \cdot 0.05 \cdot 0.95 = \mathbf{0.59}$

**Exercise: calculate the Gini index for both distributions.**

# Classification versus Regression Trees

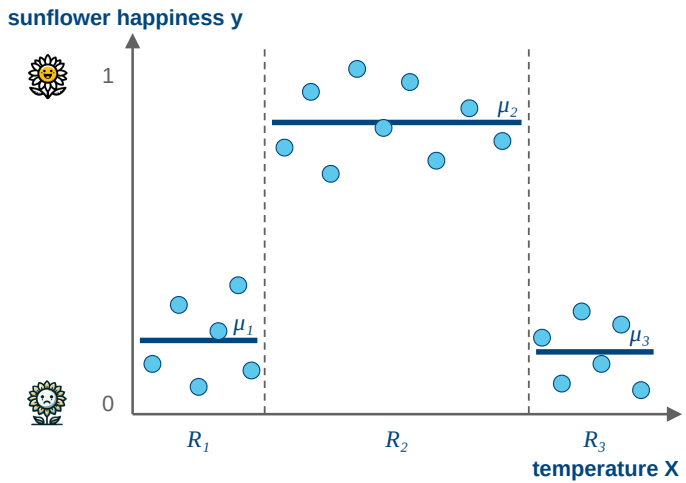
So far, we only considered **classification trees**: each leaf represents one class (the majority of the training samples in the leaf).

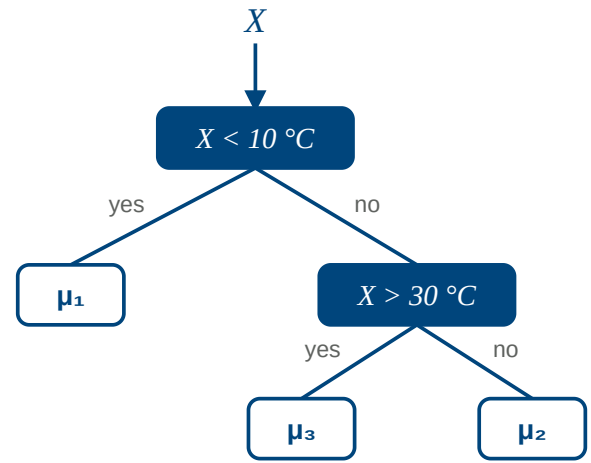
- in **classification** problems, the target variable is **categorical** (e.g., happy / not happy sunflower)
- in **regression** problems, the target variable is **continuous** (e.g., a happiness score between 0 and 1)

**Regression trees** follow exactly the same principles, but use

- a different metric for node purity: the **sample variance**
- a different interpretation of the leaves: the **sample mean**

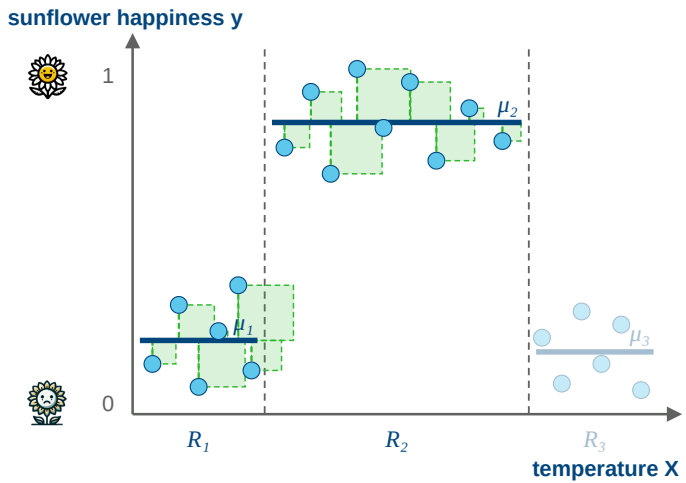
# Regression Trees: Leaves Return the Average




$$\mu_m = \frac{1}{N_m} \sum_{i:\, x_i \in R_m} y_i$$

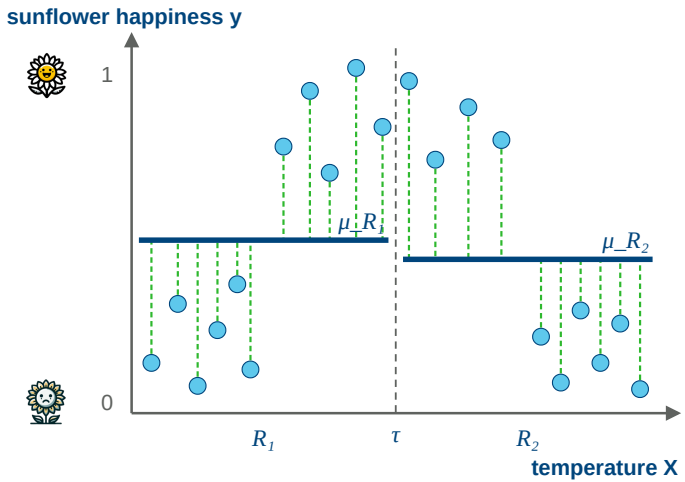
# Sample Variance as Node Purity




$$\sigma^2 = \frac{1}{N}\sum_{i=1}^{N}(y_i - \mu)^2$$

**After splitting at $10\,^\circ\mathrm{C}$: smaller average squares = lower variance = higher node purity.**

# How to Find the Threshold $\tau$?




Formal definition of the regions by the threshold

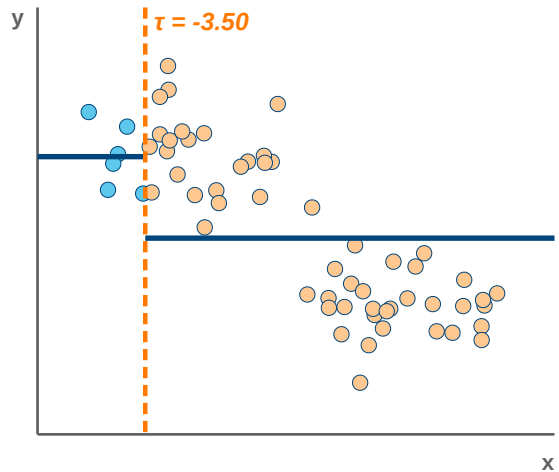
$$R_1 = \{X \mid X < \tau\}, \qquad R_2 = \{X \mid X \geq \tau\}$$

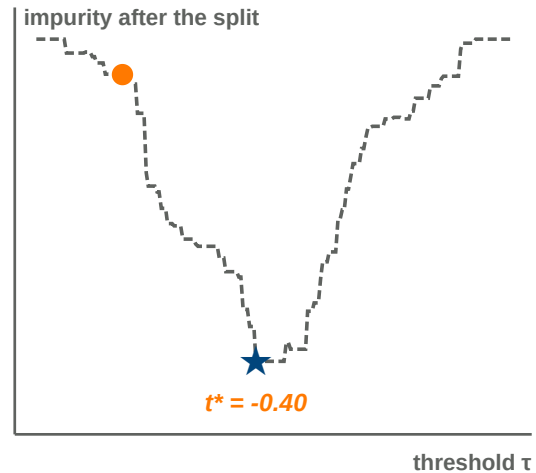
Minimization objective

$$t^* = \arg\min_{\tau} \underbrace{\sum_{i:\, x_i \in R_1} (y_i - \mu_{R_1})^2}_{\text{left variance}} + \underbrace{\sum_{i:\, x_i \in R_2} (y_i - \mu_{R_2})^2}_{\text{right variance}}$$

The impurity is a function of $\tau$ — we look for its minimum $t^*$.

# Naïve Approach: Try Different Thresholds



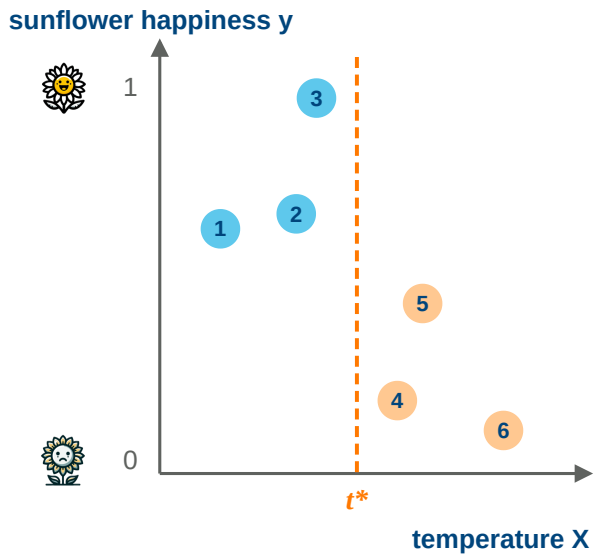




τ impurity 5.61

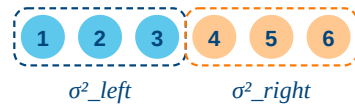
**Easy to implement — but we need to choose which thresholds to test. Drag the slider to scan the thresholds; the best recorded threshold is $t^*$.**

Original Colab notebook: threshold search

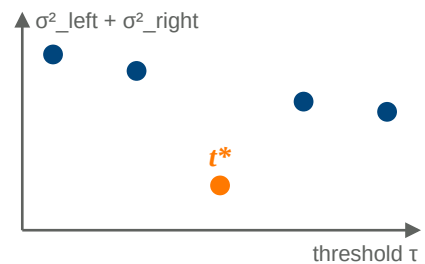
# Binary Splitting




1. sort data points by X (index 1–6)




2. for every pair of neighbours i, i+1:
   try τ between them, record σ²_left + σ²_right




**Only thresholds between neighbouring points change the split — we test as many thresholds as we have points.**

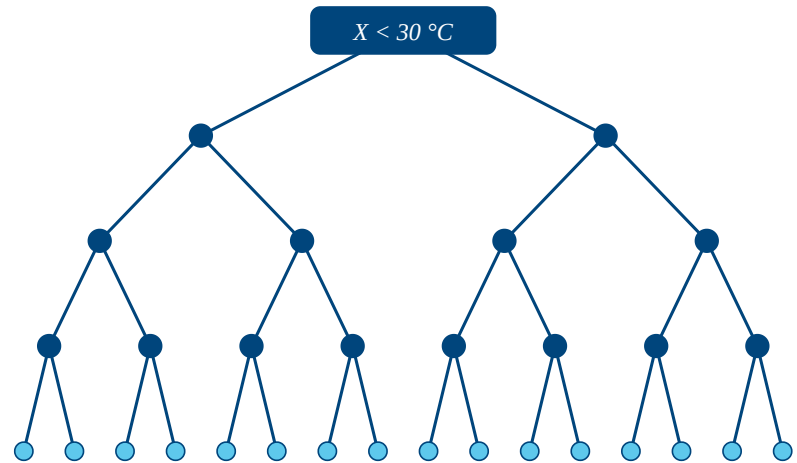
# Growing a Tree: Recursive Binary Splitting

Growing a decision tree:

- perform binary splitting
- recursively for each decision node
- to maximize node purity

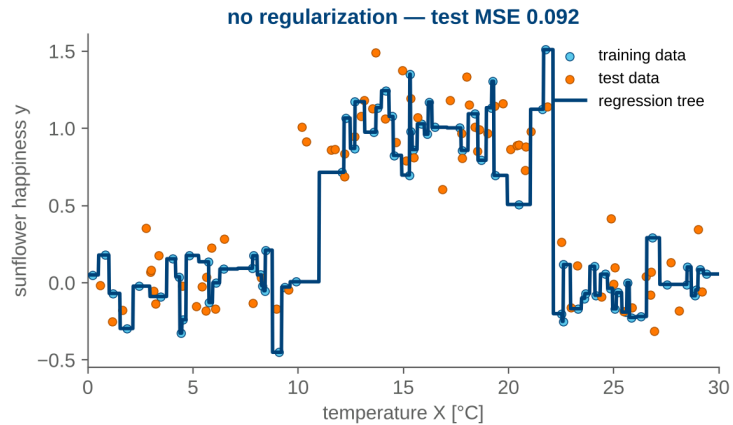
**When to stop splitting?**

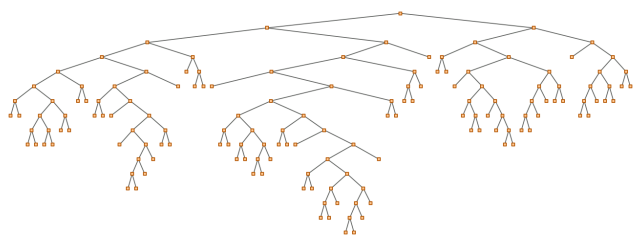
**Without a stopping rule we end up with one sample per leaf = a 1-nearest-neighbour regressor / classifier.**



# Overfitting of Decision Trees

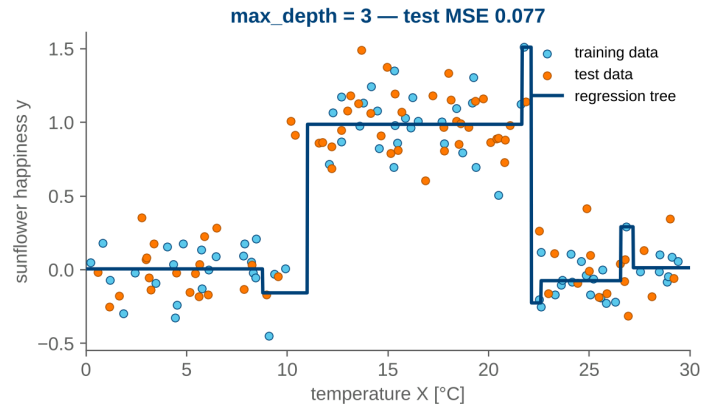
- the tree fits the training data perfectly (zero variance in every leaf)
- the tree is extremely large and uninterpretable
- test MSE: **0.092** happiness

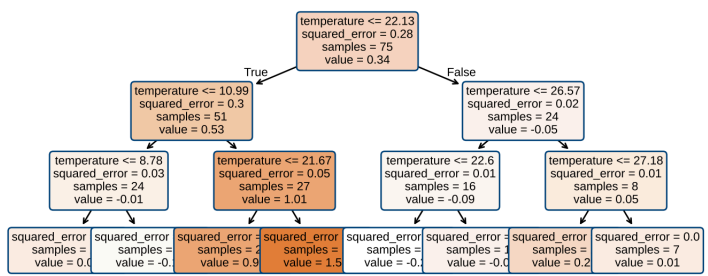


# Regularization: Restricting the Depth

- we can manually restrict the depth
- the tree remains interpretable
- test MSE lower: **0.077** happiness




Original Colab notebook: **regularization of decision trees**

# Regularization: Tree Pruning

Two families of algorithms restrict the tree complexity and counteract overfitting.

### Slow growers (pre-pruning)

**Strategy:** grow the tree slowly, with additional checks during growing.

- simple checks: **maximum depth**, **maximum number of leaf nodes**
- add a decision node only if the **increase in node purity** is above a threshold

### Fast cutters (post-pruning)

**Strategy:** grow the full tree, then remove unimportant nodes.

- **cost complexity pruning**: we identify the weakest link in the tree and remove it

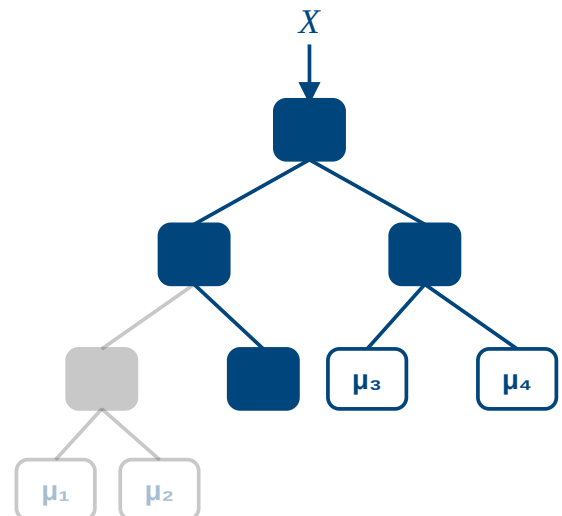
# Cost Complexity Pruning

$$C_\alpha(T) = \underbrace{C(T)}_{\substack{\text{cost} \\ \text{(training error)}}} + \alpha \underbrace{|T|}_{\substack{\text{complexity} \\ \text{(number of leaves)}}}$$

**Idea:** remove leaves to **reduce the complexity** while keeping a **low training error** (cost). The hyperparameter $\alpha$ balances both objectives.

**Algorithm:**

- iteratively remove sub-trees and record the decrease in the cost-complexity function $C_\alpha(T)$
- sub-trees can be ordered by importance, each with an associated $\alpha$



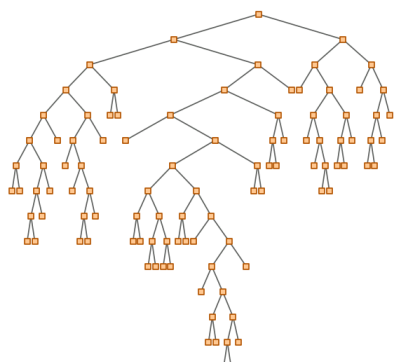
# Cost Complexity Pruning: The Effect of $\alpha$

$$C_\alpha(T) = C(T) + \alpha|T|$$

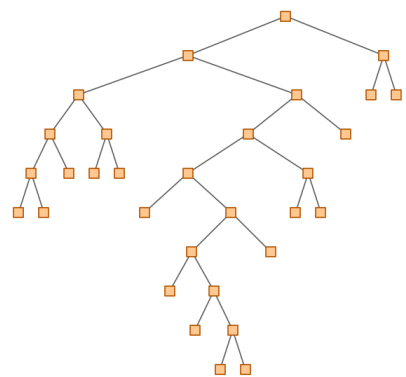
A higher $\alpha$

- produces a **shallower tree** with fewer nodes (reduces complexity)
- increases the impurity, i.e., the **training error** (increases cost)

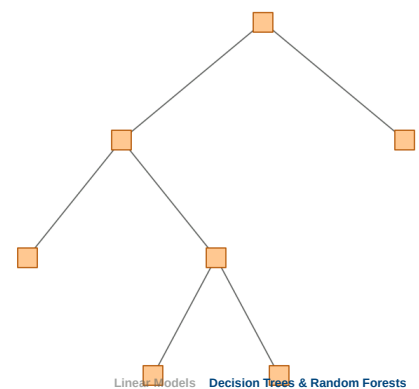
α = 0.0001
51 leaves · train MSE 0.001 · test MSE 0.092



α = 0.001
16 leaves · train MSE 0.011 · test MSE 0.079



α = 0.003
4 leaves · train MSE 0.029 · test MSE 0.061



# Building a Tree with Cost Complexity Pruning

1. use ***recursive binary splitting*** to grow a large tree on the training data
2. apply **cost complexity pruning** to the large tree to obtain a sequence of best sub-trees as a function of $\alpha$
3. use **K-fold cross-validation to choose** $\alpha$:
    1. repeat steps 1 and 2 on all but the $k$-th fold of the training data
    2. evaluate the error on the left-out $k$-th fold as a function of $\alpha$; average the results for each $\alpha$ and pick $\alpha$ to minimize the average error
4. return the sub-tree from step 2 that corresponds to the chosen value of $\alpha$



Image by Bing Image Generator

Algorithm after James, Witten, Hastie & Tibshirani, ***An Introduction to Statistical Learning***, Algorithm 8.1.

# Decision Trees in scikit-learn: Hyperparameters

```
from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor

clf = DecisionTreeClassifier(criterion="gini",          # or "entropy" — node purity
                             max_depth=None, max_leaf_nodes=None,      # pre-pruning
                             min_samples_split=2, min_samples_leaf=1,
                             min_impurity_decrease=0.0,
                             ccp_alpha=0.0)                            # post-pruning

reg = DecisionTreeRegressor(criterion="squared_error",  # variance as node purity
                            max_depth=3, ccp_alpha=0.0)
```

All hyperparameters in the snippet were covered in this part: the **purity criterion**, **pre-pruning** limits and **cost complexity pruning** via `ccp_alpha`.

Documentation: DecisionTreeClassifier · DecisionTreeRegressor

# Recap: Decision Trees

## Advantages

- **inexpensive** to construct
- extremely **fast at prediction** time
- **easy to interpret** for small trees
- **robust to noise** (when regularized)
- can easily handle **redundant or irrelevant attributes** (unless the attributes are interacting)

## Disadvantages

- decision trees are prone to **overfitting** and need regularization
- **greedy** approaches often fail to find the best tree (restricting growth is too short-sighted)
- do **not take interactions between features** into account: each decision boundary involves only a single attribute

# Bagging, Random Forests & Boosting

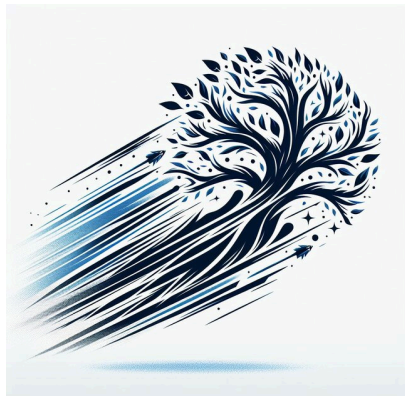
## Increasing robustness by redundancy



**Bagging**



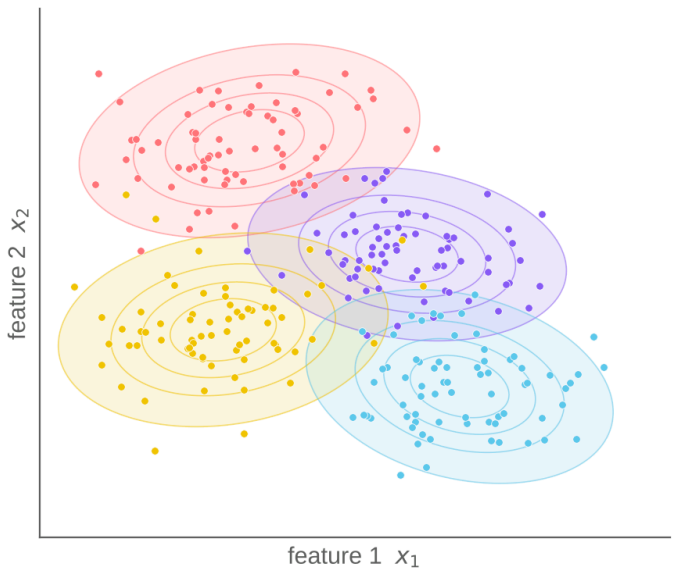
**Random Forests**



**Boosting**

Images by Bing Image Generator (DALL·E 3): "a stylized image of a bag of green trees", "… of a random forest", "… of a flying tree that gets faster and faster", white background.

# An Artificial Classification Dataset




Assume four distributions, one per class, over a random feature vector $\mathbf{X} = (x_1, x_2)$:

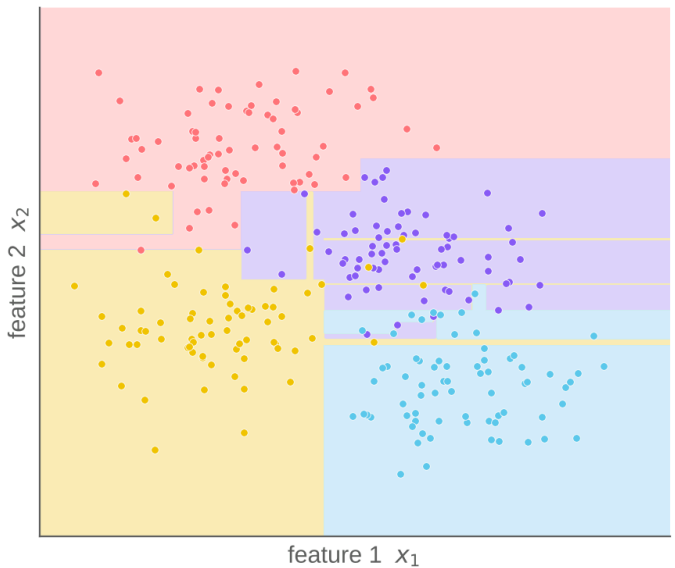
- class 1: $p(x_1, x_2 \mid y = 1)$
- class 2: $p(x_1, x_2 \mid y = 2)$
- class 3: $p(x_1, x_2 \mid y = 3)$
- class 4: $p(x_1, x_2 \mid y = 4)$

We **sample a dataset** from the distribution

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N} \sim p(\mathbf{X}, Y)$$

**Remember: we are always working with distributions.**

# Let’s Fit a Decision Tree to the Data

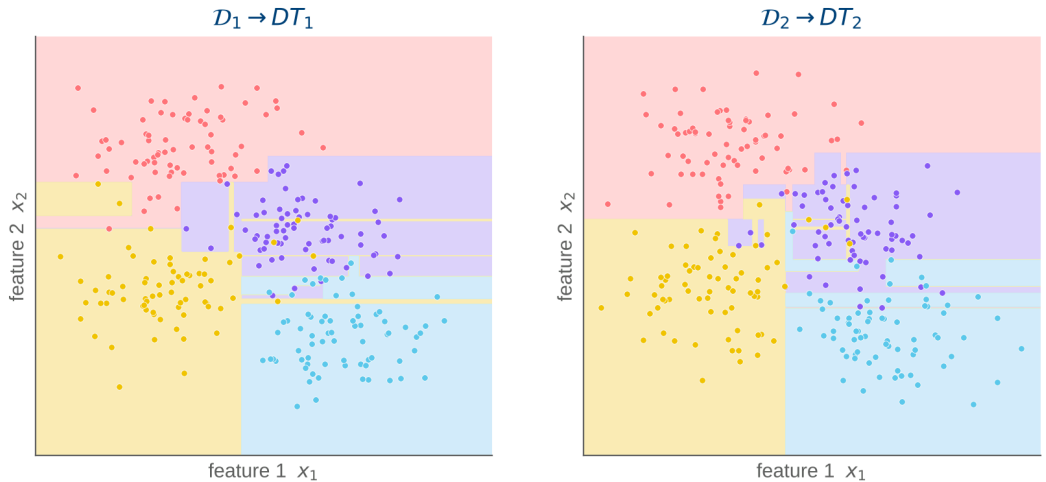



Again, we split the feature space into **regions**.

**The modes (i.e., centers) of the data distributions are well assigned to the correct classes.**

**But the uncertain, overlapping region is not well defined: thin slivers and islands follow individual training points.**

# Different Samples Lead to Very Different Trees




**Low bias: each tree fits its own training data perfectly.**

**High variance: the trees are very different on different dataset samples $\mathcal{D}_1, \mathcal{D}_2 \sim p(\mathbf{X}, Y)$.**

# Bias–Variance Trade-off

Finding a good balance between fitting and overfitting is the fundamental challenge in machine learning. For tree-like classifiers:

### Strategy 1: reduce tree complexity

(recap: seen in Part 1)

- greedy: restrict the tree complexity while growing
- non-greedy: post-hoc pruning of the tree



*"cut the tree"*

### Strategy 2: ensemble many trees

- bagging
- random forests
- boosting



*"grow a forest"*

# Let’s Grow a Forest — Step 1: Bagging

$$\mathcal{D}_{\text{train}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N} \sim p(\mathbf{X}, Y)$$

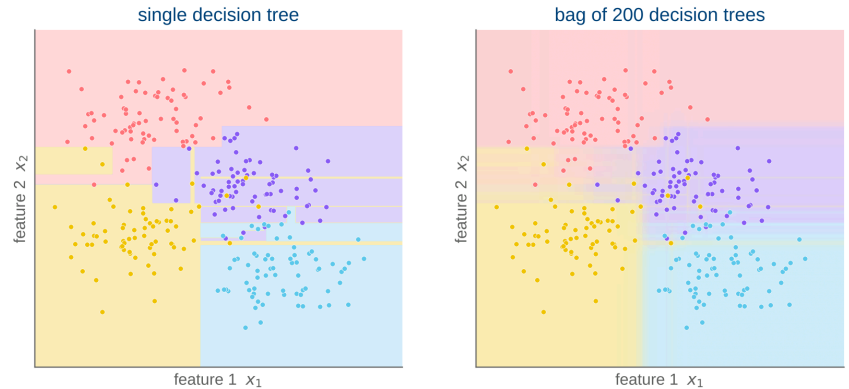
↓ randomly sampled subsets (with replacement),
i.e., a **"bootstrap"** ↓

**Bag of decision trees**



$\mathcal{D}_1 \rightarrow DT_1$ $\mathcal{D}_2 \rightarrow DT_2$ $\mathcal{D}_3 \rightarrow DT_3$

each tree is deep and not pruned

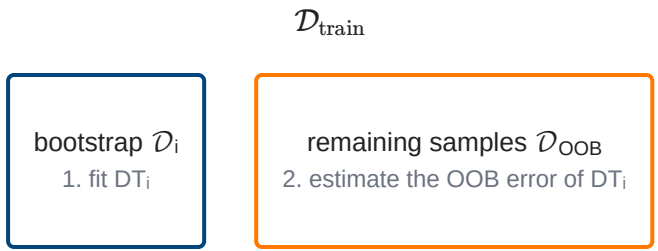



**Ensemble prediction:**

- **majority voting** for classification
- **output averaging** for regression

**Effect: trees agree in the modes and disagree in the overlap — decision boundaries become blurry, a more realistic result.**

# Bagging: Out-of-Bag Error




**Out-of-bag (OOB) error estimation**

1. fit decision tree $DT_i$ on $\mathcal{D}_i$
2. test its performance on the out-of-bag set $\mathcal{D}_{\text{OOB}}$
3. repeat for each tree in the bag
4. average the individual OOB errors across trees

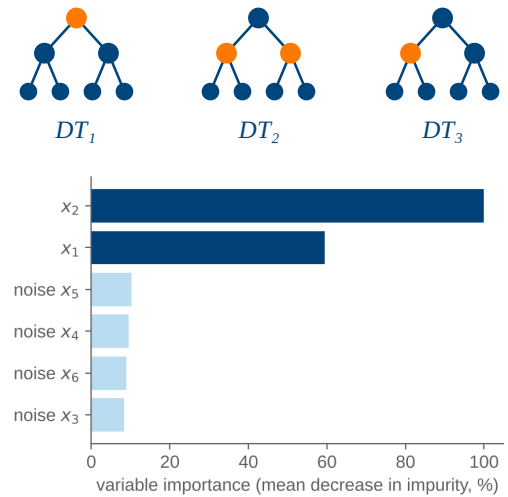
**The OOB error approximates the test error without using test data — a similar idea to cross-validation.**

# Bagging Trees: Re-gaining Interpretability

- bagging **improves accuracy** over a single tree through increasing diversity in the decisions
- but we **lose interpretability**: each tree has different decision rules, and interpreting many rules in parallel is difficult

**Variable importance** — across all trees in the bag:

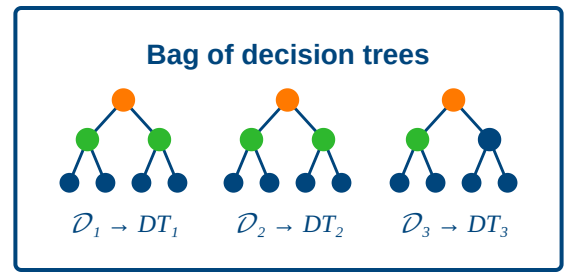
1. look at all decision nodes that involve the feature of interest ● (for instance, temperature)
2. measure the **increase in node purity** at each of these nodes and average it



# Bagging Limitation: Correlated Trees

- bagging improves accuracy through increasing **diversity** in the decisions
- but bagged trees are still **highly correlated**: an important feature ● is split first, followed by the second most important feature ●, and so on…
- all trees in a bag look similar → this **correlates the trees**, reduces diversity & test accuracy

**Idea: increase diversity by removing features from individual trees.**



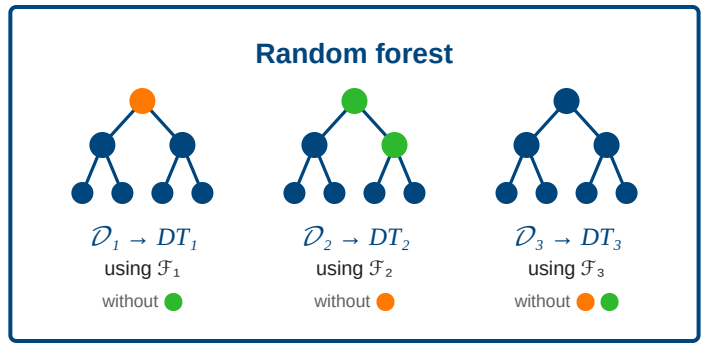
# Random Forest — Step 2: Remove Features

**Idea: increase diversity by removing features from individual trees.**

**Implementation:** we take a random sample of predictors for each tree (usually $\sqrt{F}$ of $F$ features, e.g., 4 out of 15).

In a random forest, each decision tree is

1. trained with a bootstrapped dataset $\mathcal{D}_i$
2. using a subset of features $\mathcal{F}_i$



# Summary: Bagging and Random Forests

## Bags & random forests

- each tree is trained on a different **bootstrapped** set of training data
- trees are trained separately from each other, **in parallel**




**bag of decision trees**

## Random forests

- additionally, each tree is trained with a **different subset of features**




**random forest**

# Bagging & Random Forests in scikit-learn

```python
from sklearn.ensemble import BaggingClassifier, RandomForestClassifier

bag = BaggingClassifier(estimator=DecisionTreeClassifier(),  # any base model
                        n_estimators=10, max_samples=1.0,     # number & size of bootstraps
                        bootstrap=True, oob_score=False, n_jobs=None)

rf = RandomForestClassifier(n_estimators=100, criterion="gini", max_depth=None,
                            max_features="sqrt",              # feature subset → diversity
                            bootstrap=True, oob_score=False, n_jobs=None)
```

Pass a `DecisionTreeClassifier` as `estimator` into the bagging classifier. `oob_score=True` computes the out-of-bag error; `n_jobs` trains trees in parallel.

Documentation: BaggingClassifier · RandomForestClassifier

# Random Forest versus Boosting Philosophy

### Random forest

- accuracy through **classifier diversity**
- all trees grown **in parallel**



### Boosting

- many small **"weak" classifiers**
- trees grown **sequentially**



# Boosting: Principles

- train one classifier after another
- reweight the data so that **wrongly classified samples become more important** for the next classifier

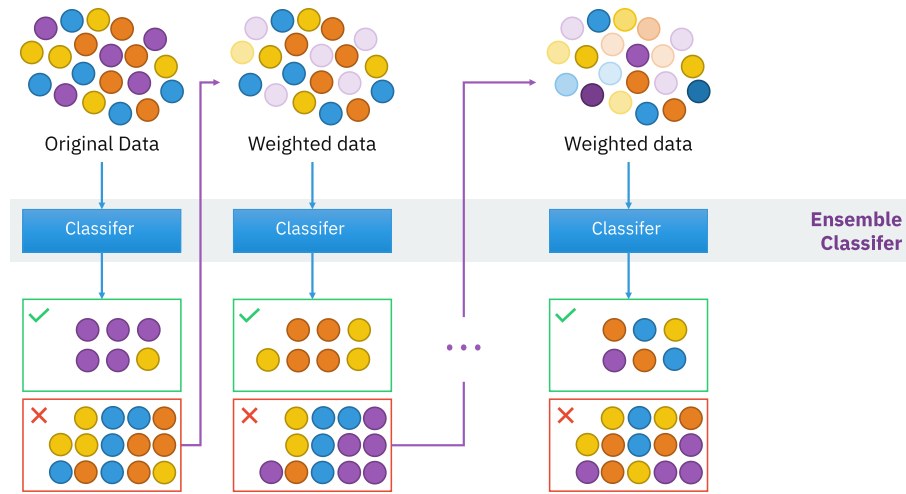
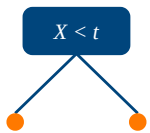



Figure: Sirakorn, Ensemble Boosting, Wikimedia Commons, CC BY-SA 4.0.

# AdaBoost: Ideas

Base classifier / regressor: a **"stump"** is a single decision node




Three core ideas:

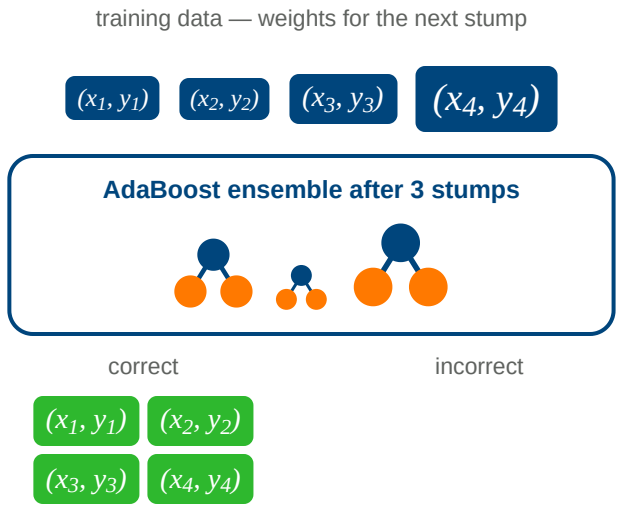
- combine many **"weak learners"**, i.e., **stumps**
- some stumps are **weighted higher** than others
- each stump takes the **previous stumps** into account



Image by Bing Image Generator

# AdaBoost: Algorithm

1. start with all data samples at **equal weight**
2. create a first stump on the feature that **increases node purity the most**
3. estimate the **error of the stump** (bad stumps with a high error contribute less)
4. make **incorrectly classified samples more important** for the next stump
5. repeat until no improvement or the maximum number of stumps is reached



# Recap: Decision Trees & Ensembles

### Decision trees

- a simple chain of decisions
- well interpretable, but inaccurate on the test set (**low bias, high variance**)

### Bag of trees

- an ensemble "bag" of trees, each trained on a different data subset ("bootstrap")
- better test accuracy through tree consensus; all trees learned **simultaneously**

### Random forest

- like a bag, but each tree also uses a different **feature subset**
- trees are as **diverse** as possible; learned simultaneously

### Boosting (AdaBoost)

- an ensemble of small trees (stumps), each trained on the **errors of the previous** ones
- trees learned **sequentially**; each additional tree "boosts" the performance